BEYOU Matthieu

LEDRU Mathieu

Extraction d'information dans les textes

Introduction

La répartition du travail pour ce projet est la suivante :
- Le système de base : Matthieu BEYOU
- Construction de classes de mots : Mathieu LEDRU
- Détérioration du système de base : Matthieu BEYOU
- Amélioration du système de base : Mathieu LEDRU

Le code, les résultats et le log des exécutions sont toutes incluses dans le dossier '20news'.

Les fichiers de code ont tous été écrit en langage perl et sont suffixé par l'extension 'pl'. Ils contiennent un commentaire au début du code qui indique la commande du terminal à écrire pour pouvoir faire tourner le script perl. Tous les fichiers de code sont normalement suffisamment commentés pour pouvoir comprendre ce que font les instructions.

Les fichiers préfixés par Qi sont liés aux questions i) du sujet. Dans la suite de ce document nous détaillerons l'utilité de chaque fichier enregistré dans le dossier '20news'.

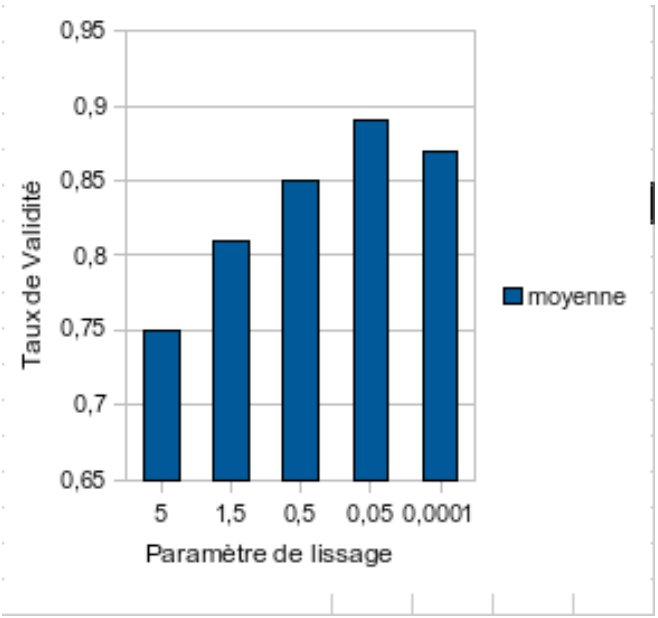
1) Le système de base

Afin d'évaluer les performances du classificateur bayésien naïf, celui-ci va débuter par une phase d'apprentissage pour un certain pourcentage de documents (définit à la main dans le programme dans la variable $pourcentageFichierAppris : ex 90) pour chaque dossier supervisé. Puis le programme va ensuite classer les documents restant non lu dans les dossiers supervisés. Ainsi on peut attribuer une note en comptant le nombre de documents bien classés sur le nombre de documents totaux à classer.

Le programme correspondant a été écrit dans le fichier 'Q1Baesien.pl'

Voici les résultats numériques pour différentes valeurs du lissage de Laplace (définit à la main dans le programme dans la variable $epsilon : ex 1.5) pour un apprentissage de 90 pourcent du corpus supervisé (note : les logs et résultats sont enregistrés dans les fichiers 'Q1terminal.txt' respectivement 'Q1resultats.ods') :

paramètre de lissage	5	1,5	0,5	0,05	0,0001
alt.atheism	0,77	0,85	0,83	0,9	0,85
comp.graphics	0,64	0,73	0,8	0,85	0,8
comp.os.ms-windows.misc	0,22	0,4	0,57	0,78	0,77
comp.sys.ibm.pc.hardware	0,81	0,85	0,86	0,86	0,81
comp.sys.mac.hardware	0,66	0,74	0,83	0,86	0,81
comp.windows.x	0,93	0,88	0,87	0,85	0,82
misc.forsale	0,46	0,56	0,73	0,8	0,69
rec.autos	0,78	0,82	0,87	0,92	0,92
rec.motorcycles	0,85	0,92	0,93	0,95	0,95
rec.sport.baseball	0,88	0,93	0,93	0,95	0,93
rec.sport.hockey	0,97	0,98	0,98	0,98	0,98
sci.crypt	0,93	0,95	0,97	0,95	0,95
sci.electronics	0,48	0,67	0,77	0,78	0,77
sci.med	0,87	0,88	0,93	0,95	0,93
sci.space	0,88	0,93	0,93	0,95	0,97
soc.religion.christian	0,9	0,9	0,9	0,87	0,85
talk.politics.guns	0,84	0,93	0,96	0,98	0,93
talk.politics.mideast	0,96	0,95	0,95	0,95	0,95
talk.politics.misc	0,94	0,94	0,96	0,98	0,98
talk.religion.misc	0,21	0,34	0,47	0,63	0,76
moyenne	0,75	0,81	0,85	0,89	0,87



On remarque qu'il existe un paramètre de lissage qui permet d'avoir un taux de validité optimal. En effet, si le paramètre de lissage venait à être trop grand, alors la log vraisemblance calculée serait mal approximée. Dans le cas contraire, si le paramètre de lissage venait à tendre vers zéro, alors on reviendrait au problème de savoir comment calculer la log vraisemblance, car les valeurs obtenues tendraient vers l'infini puisque log(0) n'existe pas.

Ici, le paramètre de lissage optimal est au alentour de 0.05. Nous choisirons le paramètre 0.05 comme le meilleur paramètre à utiliser pour ce bayésien naïf.

2) Construction de classes de mots

i) Implémentation des k-moyennes que nous avons utilisé :

Pour cette partie, les résultats ont été obtenus en exécutant le programme perl 'Q2kMoyenne.pl'. Ce qu'il faut savoir, c'est qu'il nous a fallut faire tourner le programme sur 10, 20 et 100 classes pendant environ 2 jours et demie pour pouvoir obtenir ces résultats.

Nous avons pourtant cherché à optimiser les instructions dans le code du fichier 'Q2kMoyenne.pl'. Ces quelques optimisations sont détaillées dans le code source du fichier et sont décrites par des commentaires.

ii) Autre implémentation des k-moyennes censée fournir de meilleurs résultats en temps de calcul :

Nous avons alors tenté de chercher une autre façon d'optimiser l'algorithme des k-moyennes à l'aide du document 'Q2SpeedUpKMoyenne.pdf'.

L'astuce dans cette méthode est d'utiliser des propriétés sur l'inégalité triangulaire :

Soit c1, c2 deux classes et x un mot
si d(c1, c2) / 2 >= d(x, c1) alors d(x, c2) >= d(x, c1)

Application :

Soit D(C) l'ensemble des classes privé de la classe C
si min(d(c1, D(c1))) >= d(x, c1) alors classe(x) = c1

Ce qui veut dire :
Etant donné un mot noté X assigné à une classe notée A, alors il n'est pas nécessaire de calculer toutes les autres distances de ce mot aux autres classes si la double distance entre le mot X et la classe A est inférieure au minimum de toutes les distances entre les classes.

Cette propriété permet alors d'accélérer énormément le temps de calcul de l'algorithme des k-moyennes lorsque k = 100 puisque l'on a la possibilité de sauter potentiellement 99 calcul de distance entre un mot les autres classes pour toutes les itérations.

De plus, le document précise d'autres propriétés qui permettent de ne pas calculer directement la distance entre un mot sa classe actuelle entre chaque itération permettant d'accélérer de nouveau les calculs.

Cette implémentation a été écrite dans le fichier 'Q2SpeedUpkMoyenne.pl' en utilisant simplement la propriété sur les inégalités triangulaires dans un premier temps. Mais en pratique, nous n'avons pas constaté de meilleur résultat (voire pire) en temps de calcul que l'algorithme des k-moyennes implémenté dans le i).

L'explication de ces résultats plus médiocres que précédemment sont liés à la grande dimension des vecteurs pour les mots et les moyennes, en sachant que la dimension étant égale aux nombres de documents (c'est à dire dans notre cas, nous avons 6871 document dans le corpus non supervisé, donc la dimension sera de 6871). Comme les vecteurs de mots sont souvent creux, les moyennes des classes seront en fait regroupées vers l'origine vectorielle. De ce fait, la borne minimale des distances entre les classes seront beaucoup plus petites que la double distance entre un mot et une classe. La propriété énoncée ci-dessus ne sera quasiment jamais appliquée et il ne sera pas possible de déterminer directement la classe du mot et l'algorithme devra alors calculer les 99 autres distances entre le mot et les autres classes.

Nous n'avons malheureusement pas trouvé d'algorithme plus efficace et nous avons alors lancé le long calcul décrit dans le i)

3) Détérioration du système de base

Cette partie se décrit en deux temps :

- D'abord, nous avons dupliqué le dossier '20news-bydate-supervised' par le dossier '20news-bydate-supervisedQ3'. Ensuite, nous avons écrit un script perl 'Q3RemplacerMotParClasse.pl' qui permet de remplacer chaque mot d'un corpus par une classe si le mot en question est inclus dans la classe.
En pratique : la commande 'Q3RemplacerMotParClasse.pl 20news-bydate-supervisedQ3/ Q2resultats/100classes/Q2resultatsKMoyenne.txt' va réécrire tous les fichiers contenus dans le dossier '20news-bydate-supervisedQ3' et remplacer les mots par leur classes selon le fichier 'Q2resultatsKMoyenne.txt' obtenu par l'exécution des k-moyennes de la partie 2). Les mots qui n'ont pas de classe attribué seront supprimés.

- Ensuite, nous exécutons le script 'Q3BaesienMotParClasse.pl' qui effectue le classement bayésien naïf sur le corpus '20news-bydate-supervisedQ3' par exemple. Il n'y a pas de différences fondamentales entre 'Q3BaesienMotParClasse.pl' et 'Q1Baesien.pl', car on se contente simplement de désactiver le filtrage des mots ce qui est nécessaire pour le type de corpus '20news-bydate-supervisedQ3' obtenu.

Après avoir remplacé les mots du corpus par leur classe, le bayésien naïf devient :

paramètre de lissage	5	1,5	0,5	0,05	0,0001
alt.atheism	0,6	0,6	0,6	0,6	0,6
comp.graphics	0,42	0,42	0,42	0,42	0,42
comp.os.ms-windows.misc	0,37	0,37	0,37	0,37	0,37
comp.sys.ibm.pc.hardware	0,59	0,59	0,59	0,59	0,59
comp.sys.mac.hardware	0,41	0,41	0,41	0,41	0,41
comp.windows.x	0,58	0,58	0,58	0,58	0,58
misc.forsale	0,54	0,54	0,54	0,54	0,54
rec.autos	0,55	0,55	0,55	0,55	0,55
rec.motorcycles	0,5	0,5	0,5	0,5	0,5
rec.sport.baseball	0,62	0,62	0,62	0,62	0,62
rec.sport.hockey	0,47	0,47	0,47	0,47	0,47
sci.crypt	0,58	0,58	0,58	0,58	0,58
sci.electronics	0,33	0,33	0,33	0,33	0,33
sci.med	0,53	0,53	0,53	0,53	0,53
sci.space	0,85	0,85	0,85	0,85	0,85
soc.religion.christian	0,73	0,73	0,73	0,73	0,73
talk.politics.guns	0,65	0,65	0,65	0,65	0,65
talk.politics.mideast	0,61	0,61	0,61	0,61	0,61
talk.politics.misc	0,57	0,57	0,57	0,57	0,57
talk.religion.misc	0,37	0,37	0,37	0,37	0,37
moyenne	0,54	0,54	0,54	0,54	0,54



On observe que l'on a des taux de validités bien inférieurs aux taux donnés dans les résultats de la partie 1). Ceci s'explique par le fait que nous avons d'une part pas tenu compte du TF-IDF de mots que l'on trouvait dans le corpus '20news-bydate-supervised' et que l'on ne retrouve pas dans le corpus '20news-bydate-supervisedQ3', car ces mots n'ont pas été pris en compte par l'algorithmes des k-moyennes puisqu'ils avaient une occurrence d'apparition inférieure à 5. D'autre part, les mots restant dans le corpus '20news-bydate-supervisedQ3' ont été remplacé par leur classe attribuée par l'algorithme des k-moyennes. Cette détérioration provoque alors une mauvaise estimation de la log vraisemblance et se répercute sur le classement des documents.

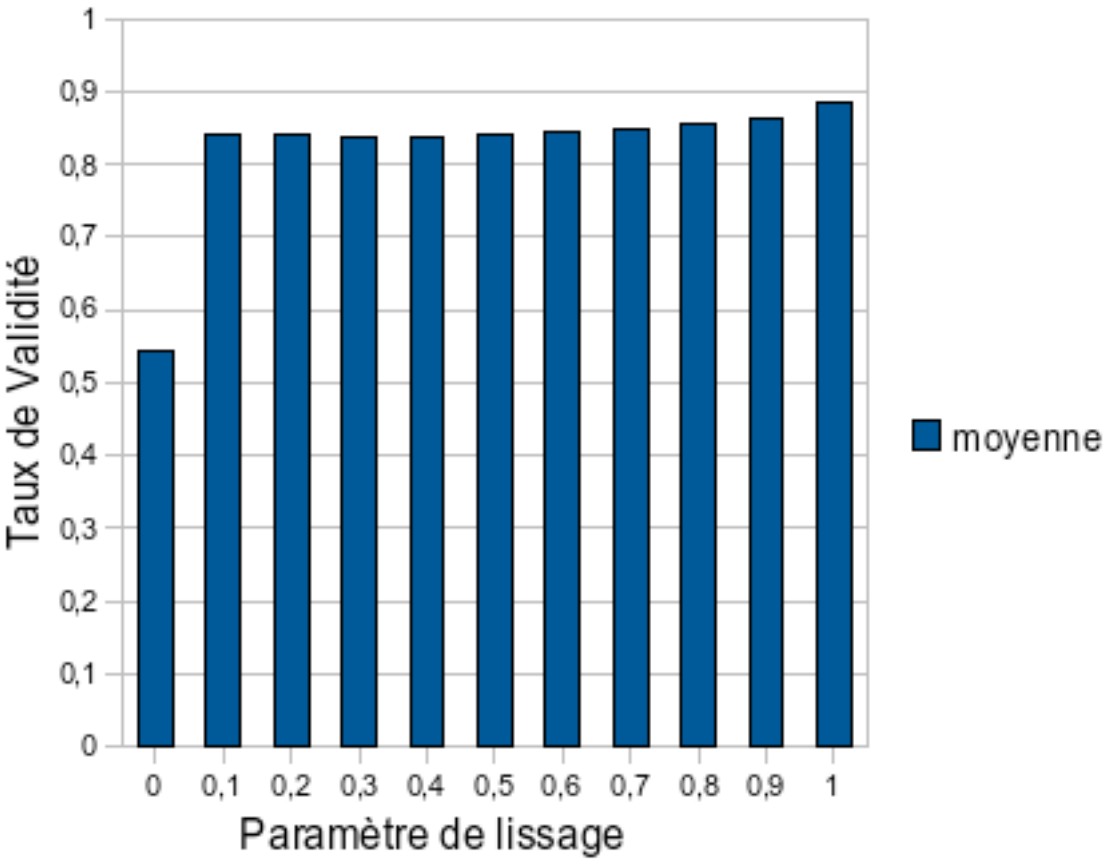
De plus, on remarque que le taux de validité ne dépend plus du paramètre de lissage de Laplace.

4) Amélioration du système de base

Maintenant, nous utilisons le classement des documents par combinaisons linéaire des deux méthodes utilisées précédemment en utilisant le programme ‘Q4BaesienMixte.pl’. Ce programme reprend exactement les codes des programmes ‘Q1Baesien.pl’ et ‘Q3BaesienMotParClasse.pl’. Cependant, la fonction d’évaluation pour les log vraisemblances a été modifiée afin de faire une combinaison linéaire entre les deux méthodes. On fait varier le paramètre alpha de 0 à 1 par intervalle de 0.1 (précisé par la variable $IAlpha au début du programme) en utilisant le meilleur paramètre de lissage que l’on a observé jusqu’à présent qui est 0.05.

Voici les résultats obtenus (note : les logs et résultats sont enregistrés dans les fichiers ‘Q4terminal.txt’ respectivement ‘Q4resultats.ods’) :

alpha	0	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8	0,9	1
alt,atheism	0,6	0,9	0,9	0,9	0,9	0,9	0,9	0,92	0,92	0,92	0,9
comp,graphics	0,42	0,73	0,73	0,73	0,73	0,73	0,73	0,73	0,75	0,75	0,85
comp,os,ms-windows,misc	0,37	0,75	0,75	0,75	0,75	0,75	0,75	0,75	0,75	0,77	0,78
comp,sys,ibm,pc,hardware	0,59	0,78	0,78	0,78	0,78	0,78	0,78	0,78	0,78	0,8	0,86
comp,sys,mac,hardware	0,41	0,76	0,76	0,76	0,76	0,76	0,76	0,78	0,79	0,81	0,86
comp,windows,x	0,58	0,78	0,78	0,78	0,78	0,78	0,8	0,8	0,8	0,82	0,85
misc,forsale	0,54	0,66	0,66	0,66	0,66	0,66	0,68	0,68	0,69	0,71	0,8
rec,autos	0,55	0,87	0,87	0,87	0,87	0,87	0,87	0,87	0,88	0,88	0,92
rec,motorcycles	0,5	0,92	0,92	0,92	0,92	0,92	0,92	0,92	0,93	0,95	0,95
rec,sport,baseball	0,62	0,93	0,93	0,93	0,93	0,93	0,93	0,93	0,93	0,93	0,95
rec,sport,hockey	0,47	0,92	0,92	0,92	0,92	0,92	0,92	0,92	0,92	0,93	0,98
sci,crypt	0,58	0,8	0,8	0,82	0,82	0,82	0,82	0,83	0,88	0,9	0,95
sci,electronics	0,33	0,77	0,77	0,77	0,77	0,78	0,78	0,8	0,82	0,82	0,78
sci,med	0,53	0,88	0,88	0,88	0,88	0,9	0,92	0,92	0,93	0,93	0,95
sci,space	0,85	0,95	0,95	0,95	0,95	0,95	0,95	0,95	0,95	0,95	0,95
soc,religion,christian	0,73	0,77	0,77	0,77	0,77	0,77	0,78	0,78	0,78	0,78	0,87
talk,politics,guns	0,65	0,91	0,91	0,91	0,93	0,93	0,93	0,93	0,95	0,95	0,98
talk,politics,mideast	0,61	0,93	0,93	0,93	0,93	0,93	0,93	0,93	0,93	0,93	0,95
talk,politics,misc	0,57	0,96	0,96	0,94	0,94	0,94	0,94	0,94	0,94	0,94	0,98
talk,religion,misc	0,37	0,84	0,84	0,82	0,82	0,82	0,82	0,82	0,82	0,82	0,63
moyenne	0,55	0,84	0,84	0,84	0,84	0,84	0,84	0,85	0,86	0,86	0,89



On remarque que pour alpha = 0 et alpha = 1, on retrouve les mêmes résultats obtenus précédemment dans les parties 3) respectivement 1).

Pour alpha compris strictement entre 0 et 1, les valeurs sont proches et légèrement inférieures à celles pour alpha = 1.